

Umsetzungs-Zeitplan und Erklärung der Umsetzung

WYSIWYG-Editor für die Anwendung „GPB Verwaltungstools“

Projektzeitraum: 24.08.2026 – 06.09.2026 · Zeitrahmen: ca. 80 Stunden

1. Übersicht des Zeitplans

Der Zeitplan verteilt die im Projektantrag festgelegten 80 Stunden auf zehn Arbeitstage innerhalb der zwei Projektwochen zu je vollen 8-Stunden-Tagen. Das letzte Wochenende dient weiterhin als Puffer für Nacharbeiten und die Abgabevorbereitung.

Datum	Phase	Thema	Aufgaben	Std.
Mo, 24.08.2026	1	Analyse & Feinkonzept	Ist-Analyse der bestehenden Tag-Button-Lösung, Anforderungsdefinition, Festlegung der Zielarchitektur	8
Di, 25.08.2026	2	Editor-Grundgerüst	HTML-Grundgerüst mit contenteditable-Fläche und Toolbar-Markup; Aufbau Selection-/Range-Zugriff	8
Mi, 26.08.2026	2	Editor-Grundgerüst	Formatierungsfunktionen Fett/Kursiv/Unterstrichen/Farbe inkl. Toggle-Logik	8
Do, 27.08.2026	2 / 3	Editor-Grundgerüst / Bild-/Anhang-Integration	Listenfunktion und Quellcode-Ansicht fertigstellen; Beginn Anhangsintegration	4 / 4
Fr, 28.08.2026	3	Bild-/Anhang-Integration	Anbindung an mini_anhang_download.php, Bildeinfügen an Cursorposition	8
Mo, 31.08.2026	4	Sanitisierung	Serverseitige Whitelist-Konzept festlegen, Sanitisierungsfunktion implementieren	8
Di, 01.09.2026	4 / 5	Sanitisierung / Testing	Validierung beim Speichern fertigstellen; Beginn Funktionstests	4 / 4
Mi, 02.09.2026	5	Testing	Browstertests (Chrome/Firefox/Edge), XSS-Testfälle gegen Sanitisierung	8
Do, 03.09.2026	6	Dokumentation	Testprotokolle abschließen; Beginn Projektdokumentation	8
Fr, 04.09.2026	6	Dokumentation	Projektdokumentation fertigstellen, Screenshots mit Testdaten erstellen	8
Sa/So, 05.–06.09.2026	–	Puffer	Reserve für Nacharbeiten, Feinschliff und Abgabevorbereitung	–
Gesamt (ohne Puffer)				80

2. Erklärung der Umsetzung je Phase

Phase 1 – Analyse & Feinkonzept (Mo, 24.08.)

Zunächst wird der bestehende Code der Tag-Button-Lösung im Mini-Kurs-Bereich (mini_kurs_sehen.php) gesichtet und dokumentiert: Welche Formatierungen werden aktuell unterstützt, wie werden sie im Textfeld erzeugt und wie wird der Inhalt gespeichert. Daraus wird das Feinkonzept für den neuen Editor abgeleitet und die Zielarchitektur festgelegt: eine contenteditable-Fläche mit eigener Toolbar auf Basis der Selection-/Range-API, ergänzt um eine serverseitige Sanitisierung beim Speichern. Als Testumgebung wird ein bestehender, bereits genutzter Testkurs im Dozentenbereich festgelegt.

Phase 2 – Editor-Grundgerüst (Di–Do, 25.–27.08.)

Das HTML-Grundgerüst besteht aus einem `<div contenteditable="true">` als Editorfläche und einer Symbolleiste mit Buttons für Fett, Kursiv, Unterstrichen, Textfarbe und Listen. Beim Klick auf einen Toolbar-Button wird die aktuelle Selektion über `window.getSelection()` ausgelesen; die Formatierung wird anschließend durch gezielte DOM-Manipulation innerhalb dieser Selektion umgesetzt (Einfügen bzw. Entfernen von `<strong>`, `<em>`, `<u>` oder `<span style="color:...">`) – bewusst ohne das veraltete `document.execCommand`. Ein Toggle-Mechanismus prüft vor jeder Aktion, ob die Selektion bereits im jeweiligen Formatierungs-Tag liegt, und hebt die Formatierung in diesem Fall auf. Listen werden über das Einfügen bzw. Auflösen von `<ul>/<ol>`- und `<li>`-Strukturen um die Selektion herum realisiert. Ergänzend wird ein Umschalter für die Quellcode-Ansicht eingebaut, der zwischen der contenteditable-Darstellung und einer `<textarea>` mit dem rohen HTML wechselt und den Inhalt bei jedem

Wechsel synchronisiert. Der zusätzliche Zeitanteil dieser Phase fließt in eine gründlichere Absicherung der Range-Logik bei verschachtelten Selektionen und Randfällen (z. B. Selektion über mehrere Blockelemente hinweg).

Phase 3 – Bild-/Anhang-Integration (Do-Fr, 27.–28.08.)

Für das Einfügen von Bildern wird der bestehende Anhangs-Upload weiterverwendet. Da die Textauswahl beim Öffnen des Upload-Dialogs verloren geht, wird die Cursorposition vor dem Upload als Range-Objekt zwischengespeichert. Nach erfolgreichem Upload liefert der Server den Dateipfad zurück, und der Editor fügt an der gespeicherten Position ein `<img>`-Tag ein, das auf den bestehenden Download-Endpunkt `mini_anhang_download.php` verweist. Der Upload-Mechanismus selbst bleibt dabei unverändert; es wird lediglich die Editor-seitige Anbindung neu geschaffen.

Phase 4 – Serverseitige Sanitisierung (Mo-Di, 31.08.–01.09.)

Beim Speichern eines Paragraphen wird der vom Editor gelieferte HTML-Inhalt vor dem Schreiben in die Tabelle `gpb_mini_paragraph` durch eine Whitelist-Sanitisierung geführt. Erlaubt sind ausschließlich die Elemente, die der Editor selbst erzeugt: `<strong>`, `<em>`, `<u>`, `<span>` mit `style`-Attribut ausschließlich für `color`, `<ul>`, `<ol>`, `<li>`, `<p>`, `<br>` sowie `<img>` mit den Attributen `src` und `alt`. Alle skriptrelevanten Attribute (z. B. `onload`, `onclick`) sowie `<script>`-, `<iframe>`- und `<object>`-Tags werden konsequent entfernt. Die Umsetzung erfolgt über HTMLPurifier oder – falls dessen Einbindung im bestehenden System nicht praktikabel ist – über eine schlanke Eigenimplementierung auf DOMDocument-Basis, die stets nach dem Prinzip „nur Erlaubtes bleibt erhalten“ arbeitet, statt einzelne gefährliche Muster zu erkennen.

Phase 5 – Testing (Di-Mi, 01.–02.09.)

Getestet werden alle Toolbar-Funktionen inklusive Toggle-Verhalten und verschachtelter Formatierungen (z. B. fett und kursiv gleichzeitig), das Bild-Einfügen sowie der Wechsel zwischen WYSIWYG- und Quellcode-Ansicht. Die Browsertests erfolgen in den aktuellen Versionen von Chrome, Firefox und Edge, jeweils in der Desktop- und der mobilen Ansicht. Ergänzend werden gezielte XSS-Testfälle gegen die Sanitisierungsfunktion durchgeführt (z. B. Einschleusen von `<script>`-Tags, Event-Handler-Attributen und verschachteltem HTML), um deren Wirksamkeit zu belegen. Alle Testfälle werden mit erwartetem und tatsächlichem Ergebnis in einem Testprotokoll festgehalten. Der zusätzliche Zeitanteil erlaubt eine breitere Abdeckung an XSS- und Randfall-Testscenarien sowie eine sorgfältigere Auswertung der Browsertests.

Phase 6 – Dokumentation (Do-Fr, 03.–04.09.)

Abschließend wird die Projektdokumentation erstellt: Ist-Analyse, Soll-Konzept, Beschreibung der technischen Umsetzung je Phase, die Testprotokolle aus Phase 5 sowie ein Fazit mit Ausblick auf eine mögliche spätere Ausrollung des Editors auf weitere Formulare im System. Ergänzt wird die Dokumentation um Screenshots der fertigen Oberfläche, die ausschließlich mit anonymisierten Testdaten erstellt werden. Die zusätzliche Zeit fließt in eine ausführlichere schriftliche Aufarbeitung der einzelnen Phasen sowie eine sorgfältigere Nachbereitung der Testprotokolle.

3. Hinweis zu den Pufferzeiten

Das abschließende Wochenende (05.–06.09.2026) ist bewusst nicht verplant und dient als Reserve, falls einzelne Phasen – insbesondere die Sanitisierung oder die Browsertests – mehr Zeit benötigen als vorgesehen. Sollte der Puffer nicht benötigt werden, kann die Zeit für eine vertiefte Dokumentation oder zusätzliche Kann-Kriterien (z. B. Undo/Redo, Tastenkombinationen) genutzt werden.